

Chapitre 2

PL/pgSQL

Programmer *dans* la base : variables, contrôle,
exceptions et curseurs

Cours intégré – préparation du TD2 / TP (chapitres 2 et 3)

Ce que SQL ne sait pas faire

SQL décrit un **résultat**. Il n'a ni variable, ni test, ni boucle. Certaines demandes ne s'expriment donc pas :

- ▶ « parcourir les films du moins cher au plus cher **et s'arrêter** quand le budget est épuisé »
- ▶ « pour chaque client, tenter un calcul, et **en cas d'erreur** passer au suivant »
- ▶ « selon le cas, faire ceci **ou** cela, puis renvoyer un message »
- ▶ « refuser automatiquement toute écriture qui violerait cette règle » (*chapitre 3*)

La bonne façon d'y penser

PL/pgSQL n'est pas un langage *concurrent* de SQL : c'est une **colle algorithmique** autour de requêtes SQL.

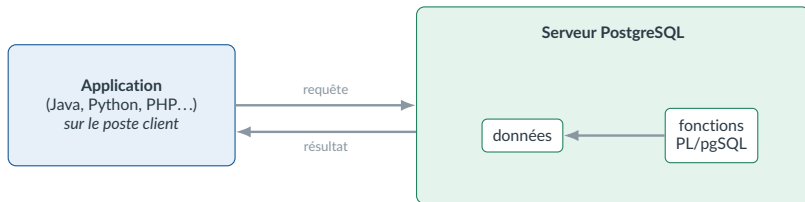
Chaque fois qu'une seule requête suffit, la requête gagne : en clarté comme en performance.

Remarque

« PL » = *Procedural Language*. Chaque SGBD a le sien : PL/pgSQL (PostgreSQL), PL/SQL (Oracle), T-SQL (SQL Server). Les idées se transposent, la syntaxe non.

Où le code s'exécute-t-il ?

la différence essentielle avec votre code Java



Conséquences

- ▶ Le code est **stocké dans la base** : il survit à l'application, et s'applique à *toutes* les applications.
- ▶ Il s'exécute à **côté des données** : pas d'aller-retour réseau par ligne traitée.

Remarque

Le revers : ce code est *invisible* depuis l'application. Un traitement qui « se produit tout seul » sera cherché longtemps du mauvais côté si personne ne sait qu'il existe.

Partie 1

Les bases du langage

déclarer, affecter, contrôler

Anatomie d'une fonction

```
CREATE OR REPLACE FUNCTION etiquette(p_id INTEGER)
RETURNS TEXT AS $$
DECLARE
    v_titre film.title%TYPE;
BEGIN
    SELECT title INTO v_titre
    FROM film WHERE film_id = p_id;
    IF NOT FOUND THEN
        RETURN 'Film inconnu';
    END IF;
    RETURN 'Film : ' || v_titre;
END;
$$ LANGUAGE plpgsql;
```

Appel : `SELECT etiquette(6);` Suppression : `DROP FUNCTION`
`etiquette(INTEGER);`

Cinq points à repérer

1. RETURNS TEXT : le type de retour est déclaré.
2. \$\$... \$\$: le corps est une *chaîne* pour PostgreSQL. On peut donc y écrire des apostrophes sans les échapper.
3. DECLARE ... BEGIN ... END : déclarations d'abord, instructions ensuite.
4. Un ; après *chaque* instruction – y compris après END IF et le END final.
5. LANGUAGE plpgsql : sans quoi PostgreSQL croit lire du SQL pur.

Déclarer des variables

DECLARE

```
v_nb      INTEGER := 0;           -- avec valeur initiale
v_total   NUMERIC;               -- vaut NULL
v_titre   film.title%TYPE;       -- type de la colonne
v_ligne   film%ROWTYPE;          -- une ligne entière
v_r       RECORD;                -- structure quelconque
c_taux    CONSTANT NUMERIC := 0.2;
```

%TYPE

Si `film.title` passe un jour de `VARCHAR(60)` à `TEXT`, la fonction suit toute seule, et le code documente d'où vient la donnée.

Piège n°1 : variable non initialisée

Une variable déclarée sans valeur vaut **NULL**, et **NULL** contamine tout :

```
v_total := v_total + 4.99;  --
                        NULL
v_res   := v_nom || ' : ';  --
                        NULL si v_nom NULL
```

Un compteur non initialisé ne vaudra jamais que **NULL**.

Affecter : deux mécanismes

Depuis une expression

```
v_nb    := 5;  
v_nb    := v_nb + 1;  
v_txt   := 'Film : ' || v_titre;
```

`:=` est la forme recommandée. PL/pgSQL tolère `=`, mais celui-ci est aussi le test d'égalité : autant lever l'ambiguïté.

Depuis la base

```
SELECT title, length INTO v_titre, v_duree  
FROM   film WHERE film_id = p_id;
```

Le `INTO` se glisse *dans* la requête, juste après la liste des colonnes.

Ce qui ne marche pas

```
v_duree := SELECT length FROM film ...;
```

syntax error at or near "SELECT"

SELECT INTO : et si la requête ne ramène pas exactement une ligne ?

le piège n°2, et le plus coûteux

La requête ramène	SELECT ... INTO v	SELECT ... INTO STRICT v
1 ligne	✓ v prend la valeur	✓ v prend la valeur
0 ligne	v devient NULL – aucune erreur	✗ exception NO_DATA_FOUND
n lignes	la première venue – aucune erreur, et l'ordre n'est pas garanti	✗ exception TOO_MANY_ROWS

Piège

Sans **STRICT**, une fonction appelée avec un identifiant inexistant ne plante pas : elle renvoie **NULL**, silencieusement.

FOUND : le témoin de la dernière requête et le piège de l'agrégation

✓ Ici, FOUND fonctionne

```
SELECT category_id INTO v_id
FROM   category WHERE name = p_nom;
IF NOT FOUND THEN
    RAISE EXCEPTION 'Catégorie inconnue : %', p_nom;
END IF;
```

La requête peut effectivement ne ramener aucune ligne.

✗ Ici, le test est du code mort

```
SELECT COUNT(*) INTO v_nb
FROM   rental WHERE customer_id = p_id;
IF NOT FOUND THEN
    RAISE EXCEPTION 'Client inexistant';
END IF;
```

La requête ne renvoie jamais aucune ligne.

Piège n°3

Une **agrégation renvoie toujours une ligne** (éventuellement 0). Donc **FOUND** vaut toujours *vrai* et l'exception n'est **jamais** levée. Dans ce cas, il faut tester directement la valeur de la variable.

Tests et branchements

IF : une instruction

```
IF v_duree IS NULL THEN
    RETURN 'durée inconnue';
ELSIF v_duree < 90 THEN RETURN 'court';
ELSE RETURN 'long';
END IF;
```

CASE : une expression (elle vaut quelque chose)

```
v_libelle := CASE v_rating
    WHEN 'G' THEN 'tous publics'
    WHEN 'R' THEN 'adultes'
    ELSE 'autre' END;
```

Piège n°4

IF v_nom = NULL n'est **jamais** vrai, même quand v_nom vaut bien **NULL**. La branche est morte, sans avertissement. On écrit IS NULL.

L'ordre des tests compte

Traitez le cas **NULL** en premier. Sinon ELSIF v_duree < 90 est faux pour une durée inconnue, et le film tombe dans le ELSE « long » (une affirmation injustifiable).

Les boucles

Boucle générale : on sort quand on veut

```
LOOP
...
EXIT WHEN condition;
END LOOP;
```

Tant que

```
WHILE v_reste >= v_tarif LOOP
    v_reste := v_reste - v_tarif;
END LOOP;
```

Compteur (i est déclaré implicitement)

```
FOR i IN 1..v_nb LOOP ... END LOOP;
FOR i IN REVERSE 10..1 LOOP ... END LOOP;
```

Sur le résultat d'une requête (curseur)

```
FOR v_r IN SELECT title, length FROM film
LOOP
    v_res := v_res || v_r.title;
END LOOP;
```

v_r doit être déclarée (de type RECORD).

Remarque

On reparlera plus en détail des curseurs un peu plus tard ! Retenez bien ce concept.

Partie 2

Renvoyer un résultat

une valeur, un ensemble, ou une table

Trois formes de retour

Déclaration	Dans le corps	Appel
RETURNS INTEGER	RETURN v; sort immédiatement	SELECT f(3);
RETURNS SETOF TEXT	RETURN NEXT v; ajoute une ligne, continue OU RETURN QUERY SELECT ...;	SELECT * FROM f(3);
RETURNS TABLE(a INT, b TEXT)	a := ...; b := ...; RETURN NEXT; idem avec a et b sont des variables OU RETURN QUERY SELECT ...;	SELECT * FROM f(3);

À retenir

Une fonction qui renvoie *une valeur* s'utilise dans le **SELECT**; une fonction qui renvoie *un ensemble* s'utilise dans le **FROM**, comme une table.

Piège

Avec RETURN QUERY, les types renvoyés doivent correspondre **exactement** à ceux déclarés :

Returned type character varying(60) does not match expected type text

RETURN contre RETURN NEXT

Historique des paiements avec cumul

```
CREATE FUNCTION cumul(p_id INTEGER)
RETURNS SETOF TEXT AS $$
DECLARE
    v_p      RECORD;
    v_cumul  NUMERIC := 0;
BEGIN
    FOR v_p IN SELECT payment_date, amount
                FROM payment
                WHERE customer_id = p_id
                ORDER BY payment_date
    LOOP
        v_cumul := v_cumul + v_p.amount;
        RETURN NEXT v_p.payment_date || ' -> ' ||
            v_cumul;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

```
SELECT * FROM cumul(5);
```

cumul
2025-02-09 -> 0.99
2025-02-16 -> 5.98
2025-02-18 -> 8.97

À retenir

RETURN NEXT **empile** une ligne et poursuit l'exécution. RETURN tout court renvoie une ligne et quitte la fonction.

Piège

Écrire RETURN au milieu d'une boucle qui devait produire N lignes est l'erreur classique : on n'en obtient qu'une.

Partie 3

Exceptions

prévenir, signaler, rattraper

Signaler et lever

```
RAISE NOTICE 'Valeur : %', v_x;  
RAISE EXCEPTION 'Client % inexistant', p_id;
```

RAISE NOTICE affiche un message (une notification), l'exécution continue

RAISE EXCEPTION interrompt et annule la transaction

Le % est le marqueur de substitution : il est remplacé par l'argument suivant.

Piège

Un RAISE NOTICE n'est **pas** une valeur de retour. Une fonction qui affiche un message mais atteint son END sans RETURN échoue : *control reached end of function without RETURN* (la notification s'affiche juste avant l'erreur).

Rattraper

```
BEGIN  
...  
EXCEPTION  
  WHEN division_by_zero THEN ...  
  WHEN no_data_found    THEN ...  
  WHEN too_many_rows    THEN ...  
  WHEN OTHERS THEN  
    RAISE NOTICE 'Erreur : %', SQLERRM;  
END;
```

À retenir

Un bloc BEGIN ... EXCEPTION ... END peut être imbriqué **n'importe où**, et notamment à l'intérieur d'une boucle : c'est ce qui permet de traiter N lignes malgré l'échec de l'une d'elles.

Où placer le bloc **EXCEPTION** ?

la position change tout

Autour de la boucle : la 1^{re} erreur arrête tout

```
BEGIN
  FOR v_c IN SELECT ... LOOP
    ... -- si ça échoue ici
  END LOOP;
EXCEPTION
  WHEN division_by_zero THEN RETURN NULL;
END;
```

... on sort de la boucle et on perd les itérations restantes.

À retenir

L'un des rares cas où le procédural est **irremplaçable**, car une requête SQL unique échoue en bloc, ou réussit en bloc.

✓ Dans la boucle : on continue

```
FOR v_c IN SELECT ... LOOP
  BEGIN
    SELECT ROUND(100.0*COUNT(return_date)/COUNT
      (*), 1)
      INTO v_taux
    FROM rental WHERE customer_id = v_c.
      customer_id;
    RETURN NEXT v_c.last_name || ' : ' || v_taux
      || ' %';
  EXCEPTION
    WHEN division_by_zero THEN
      RETURN NEXT v_c.last_name || ' : n/a';
  END;
END LOOP;
```

Un client sans location déclenche une division par zéro : absorbée, le parcours continue.

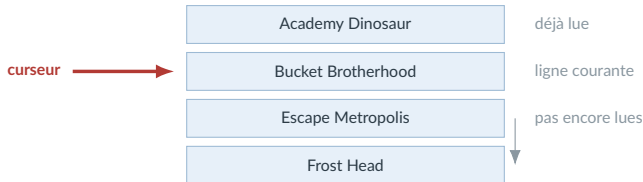
Partie 4

Curseurs

parcourir un résultat ligne à ligne

Qu'est-ce qu'un curseur ?

un pointeur qui avance sur le résultat d'une requête



À retenir

Le curseur ne charge pas tout le résultat en mémoire : il le parcourt. À chaque `FETCH`, il avance d'une ligne et la dépose automatiquement dans une variable.

Remarque

Vous en avez déjà vu un il y a quelques slides : la boucle `FOR v_r IN SELECT ... LOOP` **est** un curseur, dont l'ouverture, l'avancement et la fermeture sont automatiques. On l'appelle *curseur implicite*.

Implicite ou explicite

Curseur implicite – 90 % des cas

```
DECLARE v_r RECORD;  
BEGIN  
  FOR v_r IN SELECT title, length FROM film  
             ORDER BY title  
  LOOP  
    ...  
  END LOOP;  
END;
```

Rien à ouvrir, rien à fermer, aucun test de fin. Toutes les lignes sont parcourues.

Curseur explicite – quand il faut maîtriser l'avancement

```
DECLARE  
  c_film CURSOR FOR SELECT title, rental_rate  
                    FROM film  
                    ORDER BY rental_rate;  
  
  v_f RECORD;  
BEGIN  
  OPEN c_film;  
  LOOP  
    FETCH c_film INTO v_f;  
    EXIT WHEN NOT FOUND;  
    ...  
  END LOOP;  
  CLOSE c_film;  
END;
```

Le curseur explicite ne sert (en général) que si vous voulez **ne pas** tout parcourir : s'arrêter en route, lire une seule ligne, avancer à un rythme particulier.

Le placement du EXIT : une ligne, tout le résultat

✗ EXIT après le traitement

```
LOOP
  FETCH c_pay INTO v_p;
  v_total := v_total + v_p.amount;
  EXIT WHEN NOT FOUND;
END LOOP;
```

tour	FETCH	v_p.amount	v_total
1	✓	0.99	0.99
2	✓	4.99	5.98
3	✓	2.99	8.97
4	✗	NULL	NULL

Le FETCH qui échoue met v_p à NULL (piège n°5) : l'addition s'exécute quand même, et 8.97 + NULL vaut NULL.

✓ EXIT juste après le FETCH

```
LOOP
  FETCH c_pay INTO v_p;
  EXIT WHEN NOT FOUND;
  v_total := v_total + v_p.amount;
END LOOP;
```

À retenir

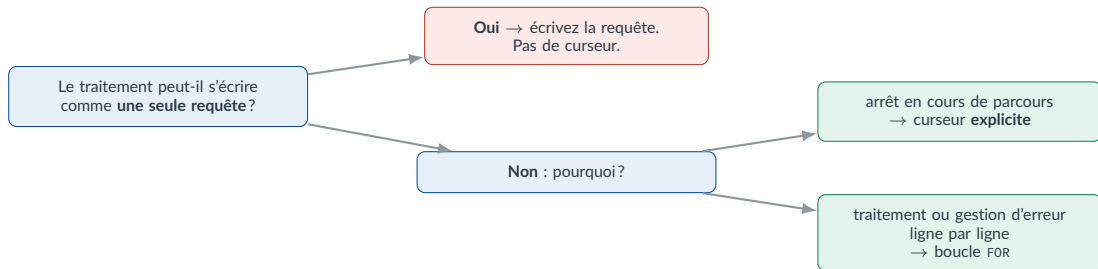
Une seule ligne déplacée, et le résultat passe de 8.97 à **NULL**, sans erreur ni avertissement.

D'où la règle suivante : **EXIT WHEN NOT FOUND colle toujours au FETCH.**

Attention, autre piège : oublier complètement le EXIT donne une boucle infinie, et le FETCH continue d'échouer sans jamais rien arrêter.

Faut-il un curseur ?

la question à se poser avant d'en écrire un



Le coût caché

Une boucle qui exécute une requête par ligne, c'est $N+1$ requêtes là où une seule suffisait. Sur 8 clients c'est indolore; sur 800 000, c'est un incident de production.

À retenir

Le SGBD est *conçu* pour traiter des ensembles. Réécrire un `GROUP BY` à la main dans une boucle, c'est refaire moins bien un travail déjà optimisé.

Partie 5

Pour conclure

toujours vivants ?



Les six pièges du chapitre

tous silencieux : aucun ne provoque de message d'erreur

1. **SELECT INTO** sans **STRICT** ne lève jamais d'erreur :
0 ligne → **NULL**, n lignes → la première venue.
2. Une variable non initialisée vaut **NULL**, et **NULL** contamine additions et concaténations.
3. **FOUND** après une agrégation vaut toujours vrai :
le test d'existence est du code mort.
4. **IF v = NULL** n'est jamais vrai. On écrit **IF v IS NULL**.
5. Un **FETCH** qui échoue met la variable à **NULL**, continuer à l'utiliser propage des **NULL**. Il faut mettre **EXIT WHEN NOT FOUND** juste après le **FETCH**.
6. Atteindre **END** sans **RETURN** est une erreur – et un **RAISE NOTICE** n'est pas un retour.

À retenir

Aucun de ces six pièges ne se voit à la lecture rapide, et aucun ne se révèle en « essayant pour voir » : la fonction s'exécute, et renvoie une valeur plausible. Il faut donc un peu réfléchir (et oui ...).

- ▶ en TP : vos propres fonctions sur la base de données `pagila` complète,
- ▶ au chapitre 3 : les **déclencheurs**, où PL/pgSQL trouve sa justification la plus forte – du code qui s'exécute tout seul pour protéger les données.

À retenir

Règle d'or : une requête si possible, une fonction si nécessaire.